CodeQuest Playground

🎮 CodeQuest Playground

Production-Grade Backend Engineering Sandbox

🧠 Project Overview

CodeQuest Playground is a **developer learning platform** where programmers solve **real-world backend engineering puzzles** instead of abstract algorithm problems.

Traditional platforms like **LeetCode** focus on algorithmic challenges such as reversing linked lists or binary trees.

CodeQuest takes a different approach.

Instead of theoretical problems, developers are dropped into **broken production-style codebases** and must debug and fix real engineering issues.

Examples include:

- Fixing a broken **JWT authentication middleware**
- Debugging a **Redis race condition**
- Solving **N+1 SQL queries**
- Fixing **CI/CD pipeline failures**
- Resolving **container deployment issues**

This creates a **hands-on backend engineering training environment**.

🎯 Core Objective

The goal of CodeQuest is to help developers learn:

- Backend debugging
- System design thinking
- Production issue diagnosis
- Infrastructure troubleshooting
- Real-world engineering workflows

Rather than learning algorithms in isolation, users learn **how real backend systems fail and how engineers fix them**.

🚀 Key Features

🧩 Context-Aware Engineering Puzzles

Each challenge simulates **real backend failures**.

Examples include:

- Incorrect route ordering causing **HTTP 404 errors**
- Broken authentication middleware
- Database query inefficiencies
- Asynchronous job queue deadlocks
- Memory leaks in backend services
- CI/CD build failures

The player must **analyze logs, inspect code, and implement fixes**.

🖥️ In-Browser Development Environment

Users interact with puzzles through an **IDE-like interface** powered by the **Monaco Editor**.

The interface provides:

- Code editor
- File explorer
- Terminal output
- Error logs
- Puzzle instructions

This replicates a **real development workflow**.

🔒 Secure Sandbox Execution

User code is executed inside **isolated containers**.

Each submission runs in a temporary container built using:

- **Docker**
- **gVisor**

Security guarantees:

- Full process isolation
- Resource limits
- Automatic container destruction
- Prevention of malicious code execution

⚡ Instant Validation

Once a solution is submitted:

1. Code is packaged by the backend orchestrator.
2. A container is spawned.
3. Hidden integration tests run.
4. The container is destroyed.
5. Results are streamed back to the UI.

This enables **real-time feedback loops** for learning.

🏗️ System Architecture

The platform follows a **microservice-inspired execution architecture** designed for safe code execution.

Layer	Technology	Responsibility
Frontend	Next.js + Tailwind	Interactive UI and puzzle interface
Code Editor	Monaco Editor	Browser-based coding environment
Orchestrator	FastAPI / Node.js	Handles puzzle state and job dispatch
Execution Engine	Docker + gVisor	Secure sandboxed code execution
State Store	Redis	Session state and rate limiting
Monitoring	Prometheus + Grafana	System performance monitoring

---

🗺️ Backend World Map (Learning Path)

The platform is structured as a **gamified world map** of backend engineering challenges.

🏁 Level 1 — Gates of HTTP

Concepts Covered

- HTTP routing
- Express middleware ordering

Puzzle

A Node.js server always returns **404 errors** even though routes exist.

Objective

Fix incorrect route ordering causing the request to never reach the correct handler.

🔐 Level 2 — Auth Dungeon

Concepts Covered

- JWT authentication
- Middleware debugging

Puzzle

The JWT token is valid, but `req.user` is always undefined.

Objective

Identify why the authentication middleware fails to attach user data.

🗄️ Level 3 — SQL Sanctum

Concepts Covered

- Database optimization
- ORM query patterns

Puzzle

A dashboard endpoint is extremely slow.

Logs show hundreds of SQL queries.

Objective

Identify and fix an **N+1 query problem**.

⚙️ Level 4 — Async Abyss

Concepts Covered

- Job queues
- Asynchronous systems

Puzzle

A worker queue using **BullMQ** becomes deadlocked.

Jobs remain stuck in the queue.

Objective

Diagnose the concurrency issue and restore job processing.

🧠 Level 5 — Infrastructure Peak

Concepts Covered

- Memory management
- Backend service stability

Puzzle

A backend API built with **FastAPI** gradually consumes all memory.

Database connection pools are exhausted.

Objective

Find unclosed database sessions causing a memory leak.

🚀 Level 6 — DevOps Trench

Concepts Covered

- Docker builds
- CI/CD pipelines

Puzzle

A deployment pipeline fails during container build.

The **Dockerfile** is incorrectly structured.

Objective

Fix a broken multi-stage Docker build to successfully deploy the service.

⚙️ Execution Workflow

The CodeQuest engine follows a **secure code evaluation pipeline**.

1️⃣ Puzzle Selection

User selects a challenge from the world map.

The platform loads:

- Broken repository
- Instructions
- Error logs
- Expected behavior

2️⃣ Code Fix

The developer edits the code inside the browser editor.

3️⃣ Submission

When the user clicks **Run**, the code is sent to the orchestrator.

4️⃣ Container Execution

The orchestrator:

1. Builds a temporary container
2. Injects the user's modified code
3. Executes hidden integration tests

5️⃣ Validation

Tests may include:

- HTTP endpoint checks
- Database query analysis
- Execution time benchmarks
- Queue processing validation

6️⃣ Results & Scoring

Results are streamed back to the frontend.

Users receive:

- Pass / Fail
- Test logs
- Debug hints
- XP points

📊 Monitoring & Observability

To maintain reliability and performance, the platform uses:

- **Prometheus** for metrics collection
- **Grafana** for visualization

Metrics tracked include:

- Container execution latency
- Failure rates
- Resource usage
- Queue processing time

🔮 Future Enhancements

Planned upgrades include:

👫 Multiplayer Debugging

Players collaborate to fix large distributed system failures.

🧠 AI Debugging Assistant

An AI system suggests hints based on stack traces and logs.

🌍 Community Challenges

Developers can upload custom puzzles based on real production issues.

🏆 Competitive Leaderboards

Rankings based on:

- Speed of fixes
- Code efficiency
- Puzzle completion streaks

📌 Why This Project Matters

CodeQuest bridges the gap between:

- Academic computer science problems

  and

- Real-world backend engineering challenges.

It trains developers to think like **production engineers**, not just algorithm solvers.